# 6A. INTRODUCTION TO PROCESSES

# THE PROCESS BLOCK

- Processes are blocks of code within the architecture
  - They are defined by the fact that statements are processed in sequence
  - This goes against the concurrent nature of VHDL
    - But is needed, especially with sequential circuits
- Process statements open significant syntax options not available concurrently
  - Syntax similar to programming allowing very flexible processing

# THE DANGER OF PROCESSES

- Processes can be difficult to write, characterize, simulate, read, and understand
- The behavior of signal assignment in processes is very confusing
- Simulation results need a thorough understanding of how signal assignments are transacted and updated
- The flexible syntax in processes may encourage bad coding
  - Unsynthesizable code
  - Even more dangerous: code that passes synthesis but produces garbage

# SYSTEMATIC USES OF PROCESSES

- Limit use of processes to avoid confusion and synthesis issues:
  - Clocked elements such as counters
  - Registers, latches, and memories
  - Selection, priority encoding, and multiplexing
  - State machines
- All syntax can be used freely and processes can be used liberally in the testbench

# VALUE EXCHANGE

```
process_label: process
begin
T <= A;
A <= B;
B <= T;
end process;
```

- The process block is delimited by begin and end process
- Statements executed in sequence
- This leads to a value exchange

# WHEN ARE PROCESSES EXECUTED

```
process_label: process
begin
T <= A;
A <= B;
B <= T;
end process;
```

- Not node connections, thus need to define when it is called
- All processes execute once at the beginning of simulation
- Processes execute again when
  - Wait statement encountered
  - Sensitivity list invoked

# SENSITIVITY LIST

```
process_label: process(R0)
begin
R1 <= R0;
R2 <= R1;
R3 <= R2;
end process;
```

- Any signal in the sensitivity list calls the process
- The call occurs at any event
  - An event is any transition on the signal

# TRACING THE PROCESS

| Time (ns) | 0 | 100 | 200 | 300 | 400 | 500 |
|---|---|---|---|---|---|---|
| R0 | 0 | 1 | 2 | 3 | 4 | 5 |
| R1 | 0 | 1 | 2 | 3 | 4 | 5 |
| R2 | 0 | 0 | 1 | 2 | 3 | 4 |
| R3 | 0 | 0 | 0 | 1 | 2 | 3 |

```
process_label: process(R0)
begin
R1 <= R0;
R2 <= R1;
R3 <= R2;
end process;
```

- This trace introduces the difference between an event and a transaction
- On each change of R0, the process is called
- Changes in all signals are transacted and take effect at the end
- Thus at 100ns, R1 updates at the end and thus R2 takes 0

# COMPLETE AND INCOMPLETE SENSITIVITY LISTS

| Time (ns) | 0 | 100 | 200 | 300 | 400 | 500 |
|---|---|---|---|---|---|---|
| R0 | 0 | 1 | 2 | 3 | 4 | 5 |
| R1 | 0 | 1 | 2 | 3 | 4 | 5 |
| R2 | 0 | 1 | 2 | 3 | 4 | 5 |
| R3 | 0 | 1 | 2 | 3 | 4 | 5 |

```
process_label: process(R0, R1, R2)
begin
R1 <= R0;
R2 <= R1;
R3 <= R2;
end process;
```

- A complete sensitivity list includes everything on the right hand of assignments, as well as operands of conditions
- It usually yields a combinational circuit
- Incomplete sensitivity lists yield strange simulations

# SYNTAX OPENED BY PROCESSES

| Statement | Notes | Use outside testbenches | Section |
|---|---|---|---|
| assert, report | Used to check status and report results | Never | 9.19 |
| Procedure call | Call functions with multiple returns | Very limited | 9.18 |
| If statement | Used for selection, conditional branching, multiplexing, | Common | 9.11 |
| Case | Selection, multiplexing, encoding. Common in state | Common | 9.11 and 9.16 |
| Loop | To create blocks that repeatedly execute | Limited | 9.19 |
| Next, exit | Interrupts execution of loops | Never | 9.19 |
| Wait | Causes execution to pause conditionally | Never | 9.19 |

# 6B. SIGNALS AND VARIABLES

# SIGNALS AND VARIABLES

- Signals form the overwhelming majority of data carriers
  - They are permanent nodes
  - Or ports
- Variables are special memory locations
  - They can be temporary
  - They are declared in a process declaration
  - They exist only in the calling process

# VARIABLE VALUES

```
process (A,B)
variable trial_var : integer := 0;
begin
C <= B;
D <= A;
E <= conv_std_logic_vector(trial_var,16);
end process;
```

- The variable can be initialized in the first process call
- On subsequent calls the variable keeps the value from the last call rather than being reinitialized
  - This can cause very complicated hardware to be synthesized

# VARIABLE AND SIGNAL ASSIGNMENT

- Variables are assigned immediately
  - When the line is encountered the value is updated
- Signal assignments within a process are only scheduled
  - When the line is encountered, there is a **transaction**
- Signal assignments are updated only at the end of the process
  - This is an event

```
process (clk)
variable var0, var1, var2 : integer := 0;
begin
if clk'event and clk = '1' then
R1 <= R0;
R2 <= R1;
var1 := var0;
var2 := var1;
end if;
end process;
```

# SIGNAL ASSIGNMENT

```
process (clk)
variable var0, var1, var2 : integer := 0;
begin
if clk'event and clk = '1' then
R1 <= R0;
R2 <= R1;
var1 := var0;
var2 := var1;
end if;
end process;
```

| Initial value | Value before end of process | Scheduled values | Final values |
|---|---|---|---|
| R0 | 5 | 5 | 5 |
| R1 | 3 | 5 | 5 |
| R2 | 10 | 3 | 3 |

- ~~Before process starts R0=5, R1=3, R2=10~~
- ~~When clk changes:~~
  - ~~Var1=var0=0~~
  - ~~Var2=new value of var1=0~~

# VARIABLE ASSIGNMENT

```
process (clk)
variable var0, var1, var2 : integer := 0;
begin
if clk'event and clk = '1' then
R1 <= R0;
R2 <= R1;
var1 := var0;
var2 := var1;
end if;
end process;
```

| ~~Initial value~~ | ~~Value before end of process~~ | ~~Scheduled~~ values | ~~Final values~~ |
|---|---|---|---|
| ~~R0~~ | ~~5~~ | ~~5~~ | ~~5~~ |
| ~~R1~~ | ~~3~~ | ~~5~~ | ~~5~~ |
| ~~R2~~ | ~~10~~ | ~~3~~ | ~~3~~ |

- Assume initialization is var0=0, var1=1, var2=2
- At clk:
  - Var1=var0=0
  - Var2=var1=var0=0, similar to variables in programming

# SIGNAL TRANSACTIONS AND EVENTS

```
process (clk)
variable var0, var1, var2 : integer := 0;
begin
if clk'event and clk = '1' then
R1 <= R0;
R2 <= R1;
var1 := var0;
var2 := var1;
end if;
end process;
```

| Initial value | Value before end of process | Scheduled values | Final values |
|---|---|---|---|
| R0 | 5 | 5 | 5 |
| R1 | 3 | 5 | 5 |
| R2 | 10 | 3 | 3 |

- Initialization R0=5, R1=3, R2=10
- At clk transact:
  - R1 = R0 = 5, R2 = R1 = 3
- Update values at end of process

# WHEN TO USE VARIABLES

- Variables can be confusing because of how they update and because their synthesis results are not always predictable
- Common usage of variables:
  - In the testbench to manage complicated test cases
  - In memories, particularly multi-port memories
  - Specific cases of synthesizable loops, especially shift registers
  - Counters

# 6C. MULTIPLEXERS AND LATCHES

# SELECTION IN A PROCESS

- Processes open two powerful syntax tools:
  - If-else statements
  - Case statements
- Both constructs are very flexible and familiar
- Special care must be taken not to accidentally create storage

# SYNTAX OF IF-ELSE STATEMENT

- Must be within a process
  - Because it is implemented serially
- The if-else statement is combinational
  - Fully populate sensitivity list
    - Including condition
- Condition can be anything that reduces to a Boolean value
  - Any combination of comparisons

```
process (A, B, F, D, E)
begin
if (A<"0001" and B>"0001") or C=F then
sigout <= D;
else
sigout <= E;
end if;
end process;
```

# MULTIPLEXING USING IF-ELSE STATEMENTS

- Synthesizes into a latch
  - Equivalent to concurrent multiplexing
- The sequential processing should modify our understanding
  - There is priority here
  - But the mutual exclusivity allows us to still consider this a MUX
- The last statement must be else to avoid creating storage
  - Will not cause failure, more dangerous

```
process (sel, A, B, C, D)
begin
if (sel = “00”) then
sigout <= A;
elsif (sel = “01”) then
sigout <= B;
elsif (sel = “10”) then
sigout <= C;
else
sigout <= D;
end if;
end process;
```

# PRIORITY ENCODING

- If the conditions in the if-else statements are not exclusive, then priority matters
- In this case notice the sequence and order of conditions

```
process (sel, sel2, A, B, C, D)
begin
if (sel = “00” and sel2=”11”) then
sigout <= A;
elsif (sel = “00”) then
sigout <= B;
elsif (sel = “10”) then
sigout <= C;
else
sigout <= D;
end if;
end process;
```

# MULTIPLEXING/ENCODING USING CASE STATEMENT

- Easier to write and read for large number of selections
- Last statement must be when others
  - For the same reason the last if statement must be an unconditional else

```
process (sel, A, B, C, D)
begin
case sel is
when "00" =>
sigout <= A;
when "01" =>
sigout <= B;
when "10" =>
sigout <= C;
when others=>
sigout <= D;
end case;
end process;
```

# NESTED CONDITIONAL STATEMENTS

- Conditional statements can be mixed and nested
- As many levels as necessary
- As long as:
  - Final else and when others statement
  - Fully populated sensitivity list

```
process (sel, sel2, sel3, A, B, C, D,E)
begin
case sel is
when '0' =>
 if sel2='0' then
   sigout <= A;
   if sel3='0' then
     sigout <= B;
   else
     sigout <= C;
   end if;
 else
 sigout <= D;
 end if;
when others=>
sigout <= E;
end case;
end process;
```

# DECLARING A LATCH IN VHDL

- Recognizing this syntax is critical
- Not because we use it
  - We generally should not
- To recognize it and remove it when found
  - Implicit latching is very dangerous

```
process (clk, D)
begin
if clk = '1' then
Q <= D;
end if;
end process;
```

# IMPLICIT LATCHING

- Very common risk of if-else and case statements
- Must never be left alone
- Does not cause synthesis errors, typically causes warnings

```
process (sel, A, B, C)
begin
if sel = “00” then
sigout <= A;
elsif sel = “01” then
sigout <= B;
elsif sel = “10” then
sigout <= C;
end if;
end process;
```

```
process (sel, A, B, C, D)
begin
if sel = “00” then
sigout <= A;
elsif sel = “01” then
sigout <= B;
elsif sel = “10” then
sigout <= C;
elsif sel = “11” then
sigout <= D;
end if;
end process;
```

# SAFE USE OF CONDITIONAL ASSIGNMENTS

```
if sel = “00” then
sigout <= A;
...
else
sigout <= D;
end if;
-----------------------------------------------------------------

case sel is
when “00” =>
sigout <= A;
..
when others =>
sigout <= D;
end case;
```

# IMPLICIT LATCHES ARE SNEAKY

- Even with a final else statement they can sneak in

```
if sel = “00” then
sig1 <= A; sig2 <= B; sig3 <= C; sig4 <= D;
elsif sel=”01” then
sig1 <= B; sig2 <= A; sig4 <= C;
elsif sel=”10” then
sig1 <= D; sig2 <= B; sig3 <= A; sig4 <= D;
else
sig1 <= B; sig2 <= C; sig3 <= C; sig4 <= C;
end if;
```

# EXPLICIT LATCHING

- Sometimes miss an assignment intentionally
- Allows signal to keep old value
- Can make code more succinct
- But better avoid it
  - Creates timing trouble
  - Readability issues
- Bottom line on latches:
  - Don't

```
case sel is
when "00" =>
sig1 <= A; sig3 <= B; sig4 <= D;
when "01" =>
sig1 <= B; sig2 <= C; sig3 <= B; sig4 <= C;
when others =>
sig1 <= A; sig2 <= D; sig3 <= B; sig4 <= D;
end case;
```

# 6D. REGISTERS AND PIPELINES

# DECLARING A FLIP FLOP

- A D flip flop is entirely declared in a single process
- The attribute ‘event is used to indicate a clock edge
  - Distinguish the positive edge
- The lack of an else in the if statement indicates storage to the synthesizer

```
entity  DFF is
port(
D : in std_logic;
Q : out std_logic;
CLK : in std_logic);
end entity;

architecture structural of DFF  is

process(CLK)
begin
if CLK’event and CLK = ‘1’ then
Q <= D;
end if;
end process;

end architecture;
```

# SYNCHRONICITY AND ENABLES

```
process(CLK)
begin
if CLK'event and CLK = '1' then
if enable = '1' then
Q <= D;
end if;
end if;
end process;
```

```
process(CLK, enable)
begin
if enable = '1' then
if CLK'event and CLK = '1' then
Q <= D;
end if;
end if;
end process;
```

- The enable signal can be checked synchronously (inside the clock condition) or asynchronously
- ~~For enables the difference is not immediately obvious because enable is usually synchronously generated~~

# RESETS

```
process(CLK)
begin
if CLK'event and CLK = '1' then
if reset = '1' then
Q <= '0';
else
Q <= D;
end if;
end if;
end process;
```

```
process(CLK, reset)
begin
if reset = '1' then
Q <= '0';
elseif CLK'event and CLK = '1' then
Q <= D;
end if;
end if;
end process;
```

- There is a huge difference in the impact of the two resets
- Always use an asynchronous reset unless you have very specific requirements
  - Asynch reset allows independent escape for all registers
  - Harmony with the SCAN path technique for DFT

# STANDARD REGISTER

- This is the standard register that should be used for all purposes
- Includes an asynchronous reset
  - Must be gathered and connected to a single global reset signal at the wrapper
- The width of the register is a generic

```
entity  Dreg is
generic(bus_width : integer := 16);
port(
D : in std_logic_vector(bus_width-1 downto 0);
Q : out std_logic_vector(bus_width-1 downto 0);
reset : in std_logic;
CLK : in std_logic);
end entity;

architecture structural of Dreg  is
process(reset, CLK)
begin
if reset = '1' then
Q <= (others => '0');
elseif CLK'event and CLK = '1' then
Q <= D;
end if;
end process;

end architecture;
```

# SHIFT REGISTER

- Shift registers are essential in aligning pipelines
- Can be declared through structural connections
- But more efficient to declare using for loops
- One of the few safe uses of for loops

```
process(reset, CLK)
begin
if reset = '1' then
shift_reg <= (others => '0');
shift_out <= (others => '0');
elsif CLK'event and CLK = '1' then
shift_out <= shift_reg(shift_reg_length-1)
for i in 0 to shift_reg_length-2 loop
shift_reg(i+1)<=shift_reg(i);
end loop;
shift_reg(0) <= shift_in;
end if;
end process;
```

# SINGLE BIT SHIFT REGISTER USING CONCATENATION

- An alternative declaration
- Simpler and easier to read
- Same synthesis results
- Works only for single bit registers

```
process(reset, CLK)
begin
if reset = '1' then
shift_out <= '0';
shift_reg <= (others => '0');
elsif CLK'event and CLK = '1' then
shift_out <= shift_reg(shift_reg_length-1);
shift_reg <= shift_reg(shift_reg_length-2 downto 0) & shift_in;
end if;
end process;
```

# A GENERAL SHIFT REGISTER

- Array user type necessary
- The width of the register is now generic

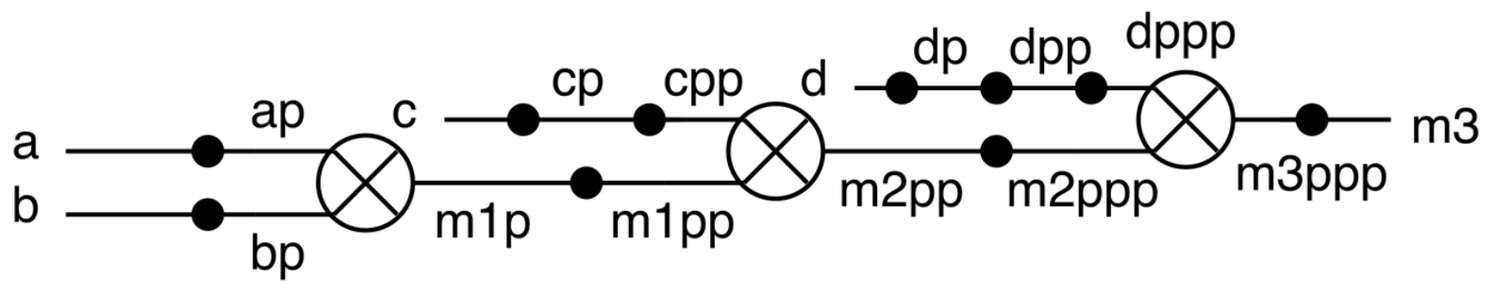
```
architecture behavioral of shift_reg  is
type r_file is array(r_length-1 downto 0) of std_logic_vector(w-1 downto 0);
signal shift_reg : r_file;
begin

process(reset, CLK)
begin
if reset = '1' then
shift_out <= (others => '0');
elseif CLK'event and CLK = '1' then
for i in 0 to r_length loop-2
shift_reg(i+1)<shift_reg(i);
end loop;
shift_reg(o)<=shift_in;
shift_out<=shift_reg(r_length-1);
end if;
end process;

end architecture;
```

# NEGATIVE EDGE TRIGGERED REGISTER

- Very simple variation on the positive register
- Do not mix clock edges in the same design
  - Can confuse tools
  - When not confusing tools, will not improve operating frequency
- Use faster clocks instead

```
process(CLK, reset)
begin
if reset = '1' then
Q <= '0';
elsif CLK'event and CLK = '0' then
Q <= D;
end if;
end if;
end process;
```

# DESIGN STRATEGY FOR PIPELINES




```
process(CLK, reset)
begin
if reset = '1' then
ap <= (others=>'0'); bp <= (others=>'0'); cp <= (others=>'0'); dp <= (others=>'0');
m1pp<= (others => '0');
m2ppp<= (others => '0');
m3 <= (others => '0');
cpp <= (others=>'0'); dpp <= (others=>'0'); dppp <= (others=>'0');
elsif CLK'event and CLK = '0' then
ap <= a; bp <= b; cp <= c; m1pp <= m1p; cpp <= cp; m2ppp <= m2pp;
dp <= d; dpp <= dp; dppp <= dpp; m3 <= m3ppp;
end if;
end process;

m1p <= ap * bp;
m2pp <= cpp * m1pp;
m3ppp <= dppp * m2ppp;
```

- Separate pipeline and data path
- Registers only in the process
- Processing in concurrent statements

# MIXED PROCESSES

- The process contains registering and DSP
- Can make code more compact
  - But does not yield better circuits
- Less readable
- Can lead to confusion about signal alignment
- Not “wrong”, but is it good?

```
process(CLK, reset)
begin
if reset = '1' then
m1 <= (others => '0');
elsif CLK'event and CLK = '0' then
m1 <= a*b;
end if;
end process;
```

# 6E. MEMORIES AND COUNTERS

# MEMORIES OR REGISTERS

- Memories are very large collections of storage with a shared port
  - Different from registers
- For FPGA, synthesizer puts them into memory blocks
- For ASICs, usually ignored and relegated to vendor specific solutions
- Memories are all about density
- Very critical for modern designs

# ENTITY DECLARATION FOR ROM

- Only has a read port
- Notice how every constant is declared as a generic

```
entity rom_block is
generic(
    address_bits : integer := 2;
    rom_word_width : integer := 4
);
port(
    clk : in std_logic;
    en : in std_logic;
    address_bus : in std_logic_vector(address_bits-1 downto 0);
    data_out : out std_logic_vector(rom_word_width-1 downto 0)
);
end entity;
```

# ROM

- The initialization of the signal rom_contents is the permanent content of the ROM
- The output is registered (synchronous)
- The output is enabled
- Arrays are essential

```
architecture behavioral of rom_block is
type rom_type is array((2**address_bits)-1 downto 0) of
    std_logic_vector(rom_word_width-1 downto 0);
signal rom_contents : rom_type := (“0000”, “0001”, “1111”, “1110”);
signal read_data : std_logic_vector(rom_word_width-1 downto 0);
begin
    read_data <= rom_contents(conv_integer(address_bus));
    process(clk)
    begin
        if clk’event and clk = ‘1’ then
            if en=’1’ then
                data_out <= read_data;
            end if;
        end if;
    end process;
end architecture;
```

# ENTITY DECLARATION OF RAM

- There can be writing or reading
- To avoid bidirectional signals (disastrously difficult to manage) have to distinct data busses
- One address means one port
- WE is used to distinguish current mode (read/write)

```
entity ram_block is
generic(
    address_bits : integer := 2;
    ram_word_width : integer := 4
);
port(
    clk : in std_logic;
    en : in std_logic;
    we : in std_logic;
    address_bus : in std_logic_vector(address_bits-1 downto 0);
    data_in : in std_logic_vector(rom_word_width-1 downto 0);
    data_out : out std_logic_vector(rom_word_width-1 downto 0)
);
end entity;
```

# ONE PORT RAM MEMORY

- Very similar to ROM architecture
- But need to check if we='1'
- If so, then the contents of the chosen location are modified

```
architecture behavioral of ram_block is
type ram_type is array((2**address_bits)-1 downto 0) of
    std_logic_vector(ram_word_width-1 downto 0);
signal ram_memory : ram_type;
begin
    process(clk)
    begin
        if clk'event and clk = '1' then
            if en='1' then
                if we='1' then
                    ram_memory(conv_integer(address_bus) <= data_in;
                end if;
                data_out <= ram_memory(conv_integer(address_bus);
                end if;
            end if;
        end if;
    end process;
end architecture;
```

# IF I AM WRITING, WHAT DO I DO ABOUT THE DOUT PORT?

```
-- Read-first method, data_out takes the value of location from previous cycle
if we='1' then
    ram_memory(conv_integer(address_bus) <= data_in;
end if;
data_out <= ram_memory(conv_integer(address_bus);
-- Latched output. The data_out port preserves its last known value
if we='1' then
    ram_memory(conv_integer(address_bus) <= data_in;
else;
    data_out <= ram_memory(conv_integer(address_bus);
end if;
-- Write first. The value of the data_out port takes the new value
if we='1' then
    ram_memory(conv_integer(address_bus) <= data_in;
    data_out <= data_in;
else;
    data_out <= ram_memory(conv_integer(address_bus);
end if;
```

# MULTI-PORT MEMORY

- Each port has a full set of busses
- Particularly, each port is characterized by an address bus
- Clocks can also be independent
  - Although this can be problematic

```
entity ram_2p is
generic(
    address_bits : integer := 2;
    ram_word_width : integer := 4
);
port(
    clk1 : in std_logic;
    en1 : in std_logic;
    we1 : in std_logic;
    address_bus1 : in std_logic_vector(address_bits-1 downto 0);
    data_in1 : in std_logic_vector(rom_word_width-1 downto 0);
    data_out1 : out std_logic_vector(rom_word_width-1 downto 0);
    clk2 : in std_logic;
    en2 : in std_logic;
    we2 : in std_logic;
    address_bus2 : in std_logic_vector(address_bits-1 downto 0);
    data_in2 : in std_logic_vector(rom_word_width-1 downto 0);
    data_out2 : out std_logic_vector(rom_word_width-1 downto 0)
);
end entity;
```

# MULTI-PORT MEMORIES AND SHARED VARIABLES

```
architecture behavioral of ram_2p is
type ram_type is array((2**address_bits)-1 downto 0) of
    std_logic_vector(ram_word_width-1 downto 0);
shared variable ram_memory : ram_type;
begin
    process(clk1)
    begin
        if clk1'event and clk1 = '1' then
            if en1='1' then
                if we1='1' then
                    ram_memory(conv_integer(address_bus1) <= data_in1;
                end if;
                data_out1 <= ram_memory(conv_integer(address_bus1);
                end if;
        end if;
    end if;
end process;
```

```
process(clk2)
begin
    if clk2'event and clk2 = '1' then
        if en2='1' then
            if we2='1' then
                ram_memory(conv_integer(address_bus2) <= data_in2;
            end if;
            data_out 2<= ram_memory(conv_integer(address_bus2);
            end if;
        end if;
    end if;
end process;
end architecture;
```

# COUNTERS ARE STRANGE

- Combine arithmetic and registering in one process
  - Contrary to common good practice
- Counters are used extensively in hardware
- Also commonly used in testbenches

# FEATURE-RICH COUNTER

- Asynchronous reset
- Synchronous enable
- Can count up or down
- ~~Can derive simpler counters by hard wiring some values~~

```
process(CLK, reset)
begin
if reset = '1' then
    counter_out <= (others => '0');
elseif CLK'event and CLK = '1' then
    if enable_counter = '1' then
        if count_up='1' then
            counter_out <= counter_out +1;
        else
            counter_out <= counter_out -1;
        end if;
    end if;
end if;
end process;
```

# COUNTER WITH LOADING

- This counter can be loaded with an external value

```
process(CLK, load_counter)
begin
elseif CLK'event and CLK = '1' then
    if load_counter = '1' then
        counter_out <= counter_in;
    else
        counter_out <= counter_out +1;
    end if;
end if;
end process;
```

# LESS THAN FULL RANGE COUNTING

- If counters are left to roll-over they count to a power of 2
- If the counter is supposed to count to a smaller figure, there must be an explicit rollover condition

```
process(CLK, reset)
begin
if reset = '1' then
    counter_out <= (others => '0');
elseif CLK'event and CLK = '1' then
    if counter_out = conv_std_logic_vector(5,3) then
        counter_out <= (others=>'0');
    else
        counter_out <= counter_out +1;
    end if;
end if;
end process;
```

# COUNTER USING VARIABLES

```
process(CLK, load_counter)
variable counter_out : integer range 0 to 7 := 0;
begin
if clk'event and clk ='1' then
    counter_out := (counter +1) mod 5;
end if;
end process;
```

- This counter uses a variable in a process to count up
- Might cause issues while trying to produce real signal outputs
- But highly useful in test benches